CECS 440 – Lab 5

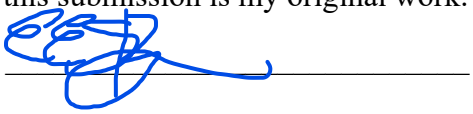
ALU and Register File

Due date: 10/16/2025

Name: Eric Santana

ID: 015107467

I certify that this submission is my original work.



Goal

The goal of this lab was to model a CPU datapath in Verilog. The main components were an Arithmetic Logic Unit, a Register File, and an Arithmetic Logic Unit Output Register. Using all these components opens the door to view how a CPU can perform arithmetic and logic operations on values stored in registers.

Steps

1. Opened Vivado and created a new project for ALU and Register File design.
2. Added the design files: alu.v, ALUout.v, regfile.v, and design.v. Imported alu.v from a previous lab.
3. Wrote and completed the Verilog code for each module according to the lab instructions.
4. Created a testbench to simulate the datapath and verify that the outputs matched the expected results for multiple operations.
5. Ran simulation in Vivado, and observed the waveform and console outputs, and confirmed proper functionality for each operation.
6. Verified that each module worked correctly and that the datapath connected to the ALU, Register FIle, and Output Register as shown in the block diagram.

Results

The ALU and Register File design functioned correctly for all test cases. Each instruction performed the correct arithmetic or logic operation based on the opcode, and the results were stored properly in the ALU output register. The simulation console and waveform confirmed the expected behavior for the operations.

Conclusion

This lab helped reinforce the concept of how a CPU can perform instructions on values stored in different registers. It showed me how data is read from a register and processed through the CPU's ALU and written back to store the result of a arithmetic or logic operation.

Appendix (1 out of 2)

Waveforms

Apendix (2 out of 3)

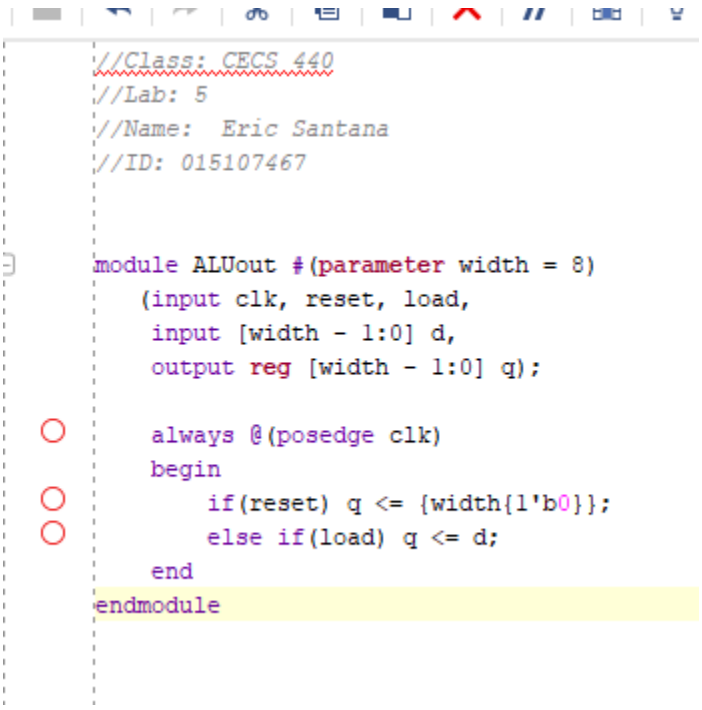
Console Output

```
Checking test case
srca =        500, srcb =       1000
add $ALUout, $ 3, $ 4 performed
sum =       1500
test passed

Checking test case
srca =        500, srcb =       1000
increment $ALUout, $ 3 performed
d_out =        501
test passed

Checking test case
srca = ffff0000, srcb = 0a0a0a0a
and $ALUout, $ 1, $ 2 performed
d_out = 0a0a0000
test passed

Checking test case
srca = ffff0000, srcb = 0a0a0a0a
or $ALUout, $ 1, $ 2 performed
d_out = ffff0a0a
test passed

Checking test case
srca = 0a0a0a0a, srcb = ffff0000
xor $ALUout, $ 2, $ 1 performed
d_out = f5f50a0a
test passed

Checking test case
srca = 0a0a0a0a, srcb = ffff0000
not $ALUout, $ 2 performed
d_out = f5f5f5f5
test passed

Checking test case
srca =       1000, srcb = 4294901760
sll $ALUout, $ 4, performed
d_out =       2000
test passed

Checking test case
srca = 000003e8, srcb = ffff0000
nop operation performed
d_out = 00000000
test passed
$finish called at time : 146 ns : File "C:/U
INFO: [USF-XSim-96] XSim completed. Design s
```

Appendix (3 out of 3)

Code

ALUout.v



```
//Class: CECS 440
//Lab: 5
//Name:  Eric Santana
//ID: 015107467


module ALUout #(parameter width = 8)
   (input clk, reset, load,
    input [width - 1:0] d,
    output reg [width - 1:0] q);

    always @(posedge clk)
    begin
        if(reset) q <= {width{1'b0}};
        else if(load) q <= d;
    end
endmodule
```

Design.v

```
//Class: CECS 440
//Lab: 5
//Name:  Eric Santana
//ID: 015107467


`include "regfile.v"
`include "ALUout.v"
`include "alu.v"

`define datasize 32

module simple_datapath
   (input [2:0] op_code,
    input clk, reset,
    input [4:0] rs, rt, rd,
    input wr_en,
    input [`datasize - 1:0] d_in,
    output [`datasize - 1:0] d_out,
    output c, n, z, p);

    wire [`datasize - 1:0] srca, srcb, aluout, aluout_y; // rfDataOut1, rfDataOut2

    regfile #(`datasize) rf
       (.clk(clk),
        .wr_en(wr_en),
        .rd_addr1(rs),
        .rd_addr2(rt),
        .wr_addr(rd),
        .wr_data(d_in),
        .rd_data1(srca),
        .rd_data2(srcb)
        ); // fill in the port list

    alu #(`datasize) a1
       (.a(srca),
        .b(srcb),
        .aluop(op_code),
        .y(aluout_y),
        .c(c),
        .n(n),
        .z(z),
        .p(p)
         ); // fill in the port list
    ALUout #(`datasize) ALUout
       (.clk(clk),
        .reset(reset),
        .load(1'b1),
        .d(aluout_y),
        .q(aluout)
           );
    assign d_out = aluout;
endmodule
```

Alu.v

```
1   `timescale 1ns / 1ps
2   //Class: CECS 440
3   //Lab: 5
4   //Name:  Eric Santana
5   //ID: 015107467
6
7
8   module alu # (parameter width = 8)
9   (
10      input [width-1:0] a,b,
11      input [2:0] aluop,
12      output reg [width-1:0] y,
13      output reg c, n, z, p
14      );
15
16  always @ (a, b, aluop)
17      begin
18      {y,c,n,z,p} = 0;
19      case(aluop)
20          0: {c,y} = a +b;
21          1:{c,y} = a + 1;
22          2:y = a&b;
23          3:y = a|b;
24          4:y = a^b;
25          5:y = ~a;
26          6:{c,y} = a <<1;
27          7:y = 0;
28          default: y = 32'bz;
29       endcase
30
31       n = y[width-1];
32       z = !y;
33       p = ^y;
34     end
35
36  endmodule
37
```

Regfile.v

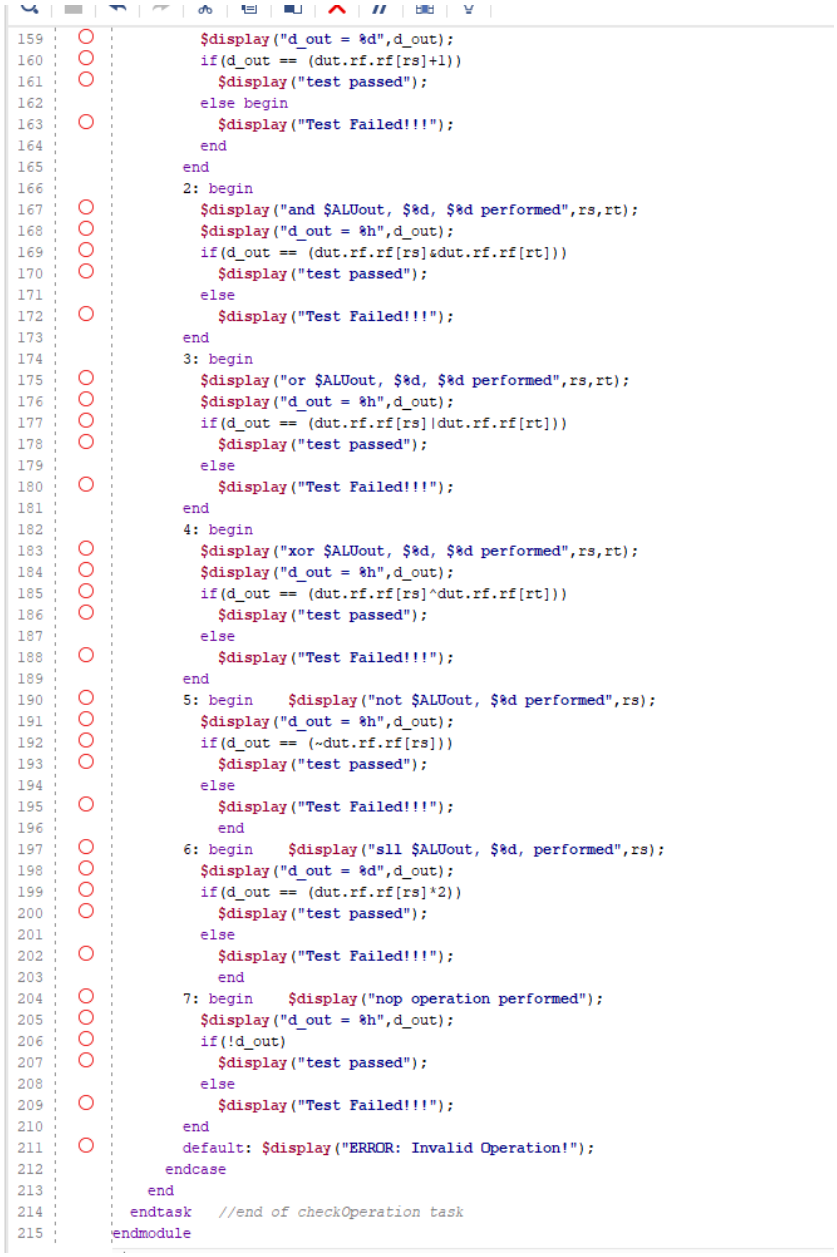
```
1  //Class: CECS 440
2  //Lab: 5
3  //Name:  Eric Santana
4  //ID: 015107467
5
6
7  module regfile #(parameter width = 8)
8     (input clk,
9      input wr_en, // write enable
10     input [4:0] rd_addr1, rd_addr2, wr_addr, // rs, rt, rd
11     input [width - 1:0] wr_data, // d_in = data in
12     output [width - 1:0] rd_data1, rd_data2); // srca, srcb
13
14 reg [31:0] rf[31:0];
15
16
17 always @(negedge clk)
18     if (wr_en) rf[wr_addr] <= wr_data;
19
20 assign rd_data1 = (rd_addr1 != 5'd0) ? rf[rd_addr1][width-1:0]:{width{1'b0}};
21 assign rd_data2 = (rd_addr2 != 5'd0) ? rf[rd_addr2][width-1:0]:{width{1'b0}};
22
23 endmodule
24
```

testbench

```
//Class: CECS 440
//Lab: 5
//Name:  Eric Santana
//ID: 015107467


module datapathTester();
  reg [2:0] op_code;
  reg clk, reset;
  reg [4:0] rs, rt, rd;
  reg wr_en;
  reg [31:0] d_in;
  wire [31:0] d_out;
  wire c,n,z,p;

  integer i;

  simple_datapath dut
   (.op_code(op_code),
    .clk(clk),
    .reset(reset),
    .rs(rs),
    .rt(rt),
    .rd(rd),
    .wr_en(wr_en),
    .d_in(d_in),
    .d_out(d_out),
    .c(c),
    .n(n),
    .z(z),
    .p(p)
    );


  always #5 clk = ~clk;

  initial begin
    op_code = 0;
    clk = 0;
    {rs, rt, rd} = 0;
    wr_en = 0;
    d_in = 0;

    reset = 0;
    #10;
    reset = 1;
    #10;
    reset = 0;
    #10;

    //load registers with values
    dut.rf.rf[0] = 0;
    dut.rf.rf[1] = 32'hFFFF0000;
    dut.rf.rf[2] = 32'h0A0A0A0A;
    dut.rf.rf[3] = 32'd500;
    dut.rf.rf[4] = 32'd1000;
    //load remaining registers with random values
```

```
    for(i = 5; i < 32; i = i + 1)
      begin
        dut.rf.rf[i] = i;
      end

    $display("Registers Initialized!");
    showRegisterContents();
    //perform ALU operations
    //Test case 1: add $ALUout, $3, $4
       @(negedge clk)
        op_code = 3'b000;
        rs = 5'b00011; // 3
        rt = 5'b00100; // 4
    @(posedge clk) #1
        checkOperation();
    //Test case 2: inc $ALUout, $3
    @(negedge clk)
        op_code = 3'b001;
        rs = 5'b00011; // 3
    @(posedge clk) #1
        checkOperation();
    //Test case 3: and $ALUout, $1, $2
    @(negedge clk)
        op_code = 3'b010;
        rs = 5'b00001; // 1
        rt = 5'b00010; // 2
    @(posedge clk) #1
        checkOperation();
    //Test case 4: or $ALUout, $1, $2
    @(negedge clk)
        op_code = 3'b011;
        rs = 5'b00001; // 1
        rt = 5'b00010; // 2
    @(posedge clk) #1
        checkOperation();
    //Test case 5: xor $ALUout, $2, $1
    @(negedge clk)
        op_code = 3'b100;
        rs = 5'b00010; // 2
        rt = 5'b00001; // 1
    @(posedge clk) #1
        checkOperation();
    //Test case 6: not $ALUout, $2
    @(negedge clk)
        op_code = 3'b101;
        rs = 5'b00010; // 2
    @(posedge clk) #1
        checkOperation();
    //Test case 7: sl $ALUout, $4
    @(negedge clk)
        op_code = 3'b110;
        rs = 5'b00100; // 4
    @(posedge clk) #1
        checkOperation();
    //Test case 8: nop
    @(negedge clk)
        op_code = 3'b111;
```

```
118
119
120     $finish;
121
122
123   end
124
125   task showRegisterContents;
126     //look directly into the register file to see contents
127     begin
128       $display("\tRegister\tContent");
129     for(i = 0; i < 32; i = i + 1)
130       begin
131         #1 $display("%d\t0x%h", i, dut.rf.rf[i]);
132       end
133     end
134   endtask   //end of showRegisterContents task
135
136   task checkOperation;
137     begin
138       $display("\nChecking test case");
139       if(op_code == 0 || op_code == 1 || op_code == 6)
140         //display arithmetic operation values in decimal
141         $display("srca = %d, srcb = %d",dut.srca, dut.srcb);
142       else
143         //display logical operation values in hex
144         $display("srca = %h, srcb = %h",dut.srca, dut.srcb);
145
146       case(op_code)
147         0: begin
148           $display("add $ALUout, $%d, $%d performed",rs,rt);
149           $display("sum = %d",d_out);
150           if(d_out == (dut.rf.rf[rs]+dut.rf.rf[rt])) begin
151             $display("test passed");
152           end
153           else begin
154             $display("Test Failed!!!");
155           end
156         end
157         1: begin
158           $display("increment $ALUout, $%d performed",rs);
159           $display("d_out = %d",d_out);
160           if(d_out == (dut.rf.rf[rs]+1))
161             $display("test passed");
162           else begin
163             $display("Test Failed!!!");
164           end
165         end
166         2: begin
167           $display("and $ALUout, $%d, $%d performed",rs,rt);
168           $display("d_out = %h",d_out);
169           if(d_out == (dut.rf.rf[rs]&dut.rf.rf[rt]))
170             $display("test passed");
171           else
172             $display("Test Failed!!!");
173         end
174         3: begin
```



```
159          $display("d_out = %d",d_out);
160          if(d_out == (dut.rf.rf[rs]+1))
161            $display("test passed");
162          else begin
163            $display("Test Failed!!!");
164          end
165        end
166        2: begin
167          $display("and $ALUout, $%d, $%d performed",rs,rt);
168          $display("d_out = %h",d_out);
169          if(d_out == (dut.rf.rf[rs]&dut.rf.rf[rt]))
170            $display("test passed");
171          else
172            $display("Test Failed!!!");
173        end
174        3: begin
175          $display("or $ALUout, $%d, $%d performed",rs,rt);
176          $display("d_out = %h",d_out);
177          if(d_out == (dut.rf.rf[rs]|dut.rf.rf[rt]))
178            $display("test passed");
179          else
180            $display("Test Failed!!!");
181        end
182        4: begin
183          $display("xor $ALUout, $%d, $%d performed",rs,rt);
184          $display("d_out = %h",d_out);
185          if(d_out == (dut.rf.rf[rs]^dut.rf.rf[rt]))
186            $display("test passed");
187          else
188            $display("Test Failed!!!");
189        end
190        5: begin    $display("not $ALUout, $%d performed",rs);
191          $display("d_out = %h",d_out);
192          if(d_out == (~dut.rf.rf[rs]))
193            $display("test passed");
194          else
195            $display("Test Failed!!!");
196            end
197        6: begin    $display("sll $ALUout, $%d, performed",rs);
198          $display("d_out = %d",d_out);
199          if(d_out == (dut.rf.rf[rs]*2))
200            $display("test passed");
201          else
202            $display("Test Failed!!!");
203            end
204        7: begin    $display("nop operation performed");
205          $display("d_out = %h",d_out);
206          if(!d_out)
207            $display("test passed");
208          else
209            $display("Test Failed!!!");
210        end
211        default: $display("ERROR: Invalid Operation!");
212      endcase
213    end
214  endtask   //end of checkOperation task
215 endmodule
```

Use of AI Tools:

ChatGPT Edu (GPT-5) was used to proofread the lab report for clarity and formatting.